

UFCFAF-30-3

Development of Information Systems Project

PORTFOLIO SUBMISSION

Group 5

GROUP MEMBERS

Sufyaan Baz
Roland Dalea
James Hayes
Wageesha Naiduwadura
Si Soe

REPORT CONTRIBUTORS

Roland Dalea
James Hayes

Table Of Contents

Forms.....	3
Validation.....	3
UX Design.....	8
Testing.....	10
Automated Testing.....	10
Worker Unit Testing.....	10
Repository Testing.....	10
REST API Controller Testing.....	11
Testing Outcome.....	11
Manual Testing.....	12
Tests done.....	12
Testing Outcome.....	12
Configuration Tools and Practices.....	13
Configuration Management.....	13
Version Control and Repository Structure.....	13
Version Tracking and Collaboration Evidence.....	13
Dependency and Environment Management.....	13
Documentation and Onboarding.....	14
Test and Demo Data Management.....	14
Automated Testing and CI Integration.....	14
Overall Evaluation.....	14
Appendix.....	16
Appendix A - All Form Validation Use.....	16

Forms

Validation

The Camunda forms developed for this project make extensive use of input validation to improve data quality and reduce the likelihood of incorrect or incomplete process submissions. Validation was implemented directly within the form definitions using Camunda's built-in validate configuration options. Several different validation techniques were used depending on the type of data being collected.

These included required field validation to ensure mandatory information is always entered, regular expression (regex) validation to enforce structured formats such as email addresses, tracking numbers, and employee IDs, numerical validation to prevent invalid quantities or negative values, and minimum/maximum length validation to restrict excessively short or long text input. Together, these validation rules help maintain process integrity and reduce the need for additional error handling within backend workers.

The following examples demonstrate how each type of validation was implemented within the project's Camunda forms.

For this example, the Batch Submission Form uses several validation rules to make sure the submitted batch data is complete, correctly formatted, and suitable for processing.

Form Preview

Batch Type*
Financial

Client's Batch Reference Number*
123-!@£\$%^&*
Field must match pattern ^[A-Za-z0-9\.-]+\$.

Notes or Comments

File Information

Record Count*
-5
For validation purposes
Field must have minimum value of 0.

Control Total / Amount
\$ -5
For financial batches
Field must have minimum value of 0.

Submitter Information

Client ID / Name
aaaaaaaaaaaaaaaaaaaaaaaa
Field must have maximum length of 255.

Submitted By (User Name)
aaaaaaaaaaaaaaaaaaaaaaaa
Field must have maximum length of 255.

Submission Date
dd/mm

Time
hh:mm

System Tracking Data

System Batch ID
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
Field must have maximum length of 255.

Batch Status
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
Field must have maximum length of 255.

Example of Batch Submission Form rejecting invalid input

The Batch Type field uses required: true. This means the user must choose a batch type before the form can be submitted. This is important because the later process flow may depend on whether the batch is financial, a data update, or an onboarding batch.

The Client's Batch Reference Number field uses multiple validation rules. The required: true rule ensures that every batch has a client-provided reference number. The minLength: 1 rule prevents the field from being submitted as empty, while maxLength: 50 prevents overly long references from being entered. The regex pattern `^[A-Za-z0-9\\-]+$` restricts the reference to letters, numbers, and hyphens only. This helps prevent invalid characters and keeps the reference in a clean ID-style format.

The Record Count field uses required: true and min: 0. This ensures that the number of records in the batch is always provided and cannot be negative. A negative record count would not make sense in the context of a submitted file, so this protects the process from invalid numeric input.

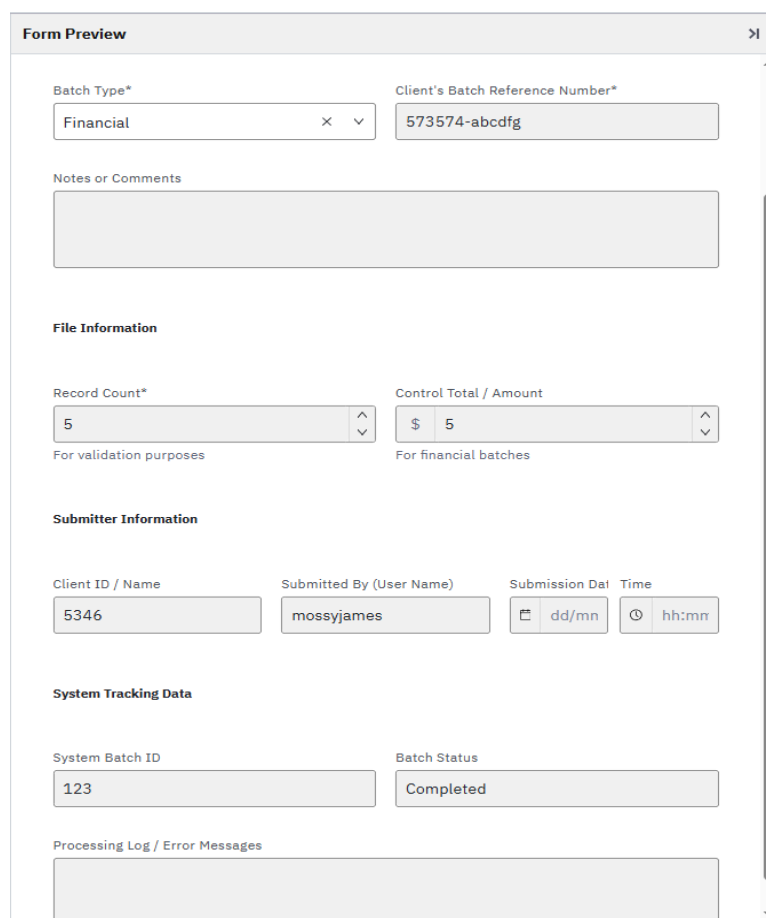
The Control Total / Amount field uses min: 0. This prevents the user from entering a negative financial total. The field is not marked as required because it may only be relevant for financial batches, but when a value is entered, it must be zero or above.

The Client ID / Name field uses maxLength: 255. This prevents the user from entering excessively long client details, while still allowing enough space for either a client ID or a full client name.

The Submitted By (User Name) field also uses maxLength: 255. This keeps the submitter name within a sensible length and prevents unusually long text being passed into the process variables.

The System Batch ID field uses maxLength: 255. This limits the length of the system-generated or manually entered batch identifier, helping keep tracking data manageable.

The Batch Status field uses maxLength: 255 and has a default value of Received. The default value gives the batch an initial process status, while the length validation prevents invalidly long status text.



The image shows a 'Form Preview' window for a 'Batch Submission Form'. The form is organized into several sections:

- Batch Type***: A dropdown menu with 'Financial' selected.
- Client's Batch Reference Number***: A text input field containing '573574-abcdefg'.
- Notes or Comments**: A large, empty text area.
- File Information**:
 - Record Count***: A numeric input field with '5' entered, with a note 'For validation purposes' below it.
 - Control Total / Amount**: A field with a currency symbol '\$' and the value '5', with a note 'For financial batches' below it.
- Submitter Information**:
 - Client ID / Name**: A text input field with '5346'.
 - Submitted By (User Name)**: A text input field with 'mossyjames'.
 - Submission Date**: A date picker showing 'dd/mn'.
 - Time**: A time picker showing 'hh:mm'.
- System Tracking Data**:
 - System Batch ID**: A text input field with '123'.
 - Batch Status**: A dropdown menu with 'Completed' selected.
- Processing Log / Error Messages**: A large, empty text area at the bottom.

Example of Batch Submission Form accepting valid input

For the next example, we will go over the Request Finance Options at Checkout Form which uses more specialised validation rules to ensure that customer contact details, identity information, and financial data are entered in a valid and consistent format before the finance request is processed.

First Name*	Last Name*
abcd1235	abcd1235
Field must match pattern <code>^[A-Za-z][A-Za-z\s\-']{1,99}\$</code> .	Field must match pattern <code>^[A-Za-z][A-Za-z\s\-']{1,99}\$</code> .

Date of Birth*
05/02/2004

Social Security Number / National ID*
<input \$%"="" type="text" value="abcd!!!!@@"/>
Field must match pattern <code>^[A-Za-z0-9][A-Za-z0-9\-_/\#]{1,59}\$</code> .

Contact Information

Email Address*	Mobile Phone Number*
<input type="text" value="sdfsd@1112343!" £\$£%"asd"=""/>	<input type="text" value="asdfs!" £\$%\$e^&\$%&\$%&*\$%*"=""/>
Field must match pattern <code>^[^\s@]+\s+@^\s@]+\.</code> <code>^[^\s@]+\$</code> .	Field must match pattern <code>^[+]?[0-9\s()-]{7,20}\$</code> . Field must have maximum length of 20.

Street Address*
1
Field must have minimum length of 5.

City*	State*	ZIP Code*
1	2	3
Field must have minimum length of 5.	Field must have minimum length of 5.	Field must match pattern <code>^[A-Za-z0-9\s\-_]{3,12}\$</code> . Field must have minimum length of 3.

Financial Information

Employment Status*

Employer Name

Field must have maximum length of 255.

Annual Income*

Field must have minimum value of 0.

Order Information

Total Purchase Amount*

Field must have maximum value of 100000000.

Agreements

☐ I consent to a credit check.*

☐ I agree to the Terms and Conditions.*

☐ I agree to the Privacy Policy.*

Submit

Example of Request Finance Options at Checkout Form rejecting invalid input

The First Name and Last Name fields are both required. They use `minLength: 2` and `maxLength: 255`, meaning the user must enter at least two characters and cannot enter an excessively long value. They also use the regex pattern `^[A-Za-z][A-Za-z\s\-\']{1,99}$`. This means the name must start with a letter and can then contain letters, spaces, hyphens and apostrophes. This allows realistic names such as “Anne-Marie” or “O'Connor”, while rejecting numbers and unusual symbols.

The Date of Birth field uses `required: true`. This ensures that the applicant provides a date of birth before submitting the finance request, as age and identity details are important for finance-related processing.

The Social Security Number / National ID field is required and uses `minLength: 2`, `maxLength: 60` and the regex pattern `^[A-Za-z0-9][A-Za-z0-9\-\_/#]{1,59}$`. This means the value must start with a letter or number and can then contain letters, numbers, hyphens, underscores, slashes, hash symbols and spaces. This gives flexibility for different national ID formats while still preventing completely unstructured text or unsupported symbols.

The Email Address field is required, limited to `maxLength: 255`, and uses the pattern `^[^\s@]+\@[^\s@]+\.[^\s@]+$`. This checks that the email contains text before and after an `@` symbol, followed by a domain section containing a full stop. It prevents common mistakes such as missing the `@` symbol or entering spaces in the email address.

The Mobile Phone Number field is required and uses `minLength: 7`, `maxLength: 20` and the pattern `^[+]?[0-9\s()]{7,20}$`. This allows phone numbers with digits, spaces, brackets, hyphens and an optional leading plus sign for international numbers. It prevents normal text from being entered while still allowing common phone number formats.

The Street Address, City and State fields are all required. The street address must be at least 5 characters and no more than 500 characters. The city and state fields also use minLength: 5 and maxLength: 500. These rules prevent blank or extremely short address values, while also limiting overly long input.

The ZIP Code field is required and uses minLength: 3, maxLength: 12 and the pattern `^[A-Za-z0-9\\s\\-]{3,12}$`. This allows letters, numbers, spaces and hyphens, making it flexible enough for different postcode or ZIP code formats while rejecting unusual characters.

The Employment Status field is required. Because it is a select field, the user must choose one of the predefined options: employed, self-employed, unemployed, student or retired. This keeps the submitted employment data consistent.

The Employer Name field is optional but has maxLength: 255. This means the user can leave it blank, but if they enter an employer name, it cannot be excessively long.

The Annual Income field is required and uses min: 0 and max: 100000000. This prevents negative income values and also sets an upper limit to avoid unrealistic or accidental entries.

The Total Purchase Amount field is also required and uses min: 0 and max: 100000000. This ensures the requested finance amount cannot be negative and stays within a controlled numeric range.

The Credit Check Consent, Terms and Conditions, and Privacy Policy checkboxes are all required. This means the user must explicitly confirm these agreements before the form can be submitted. This is important because the finance process should not continue unless the applicant has agreed to the required checks and policies.

First Name*	Last Name*	
James	Hayes	
Date of Birth*		
05/02/2004		
Social Security Number / National ID*		
1234-5678-91011		
Contact Information		
Email Address*	Mobile Phone Number*	
james2.hayes@live.uwe.ac.uk	07123456789	
Street Address*		
The Street		
City*	State*	ZIP Code*
The City	The State	S11 2AB

Financial Information

Employment Status*	Employer Name
Self-Employed × ▾	The Employer

Annual Income*
\$ 50000 ^ ▾

Order Information

Total Purchase Amount*
\$ 100 ^ ▾

Agreements

- ☒ I consent to a credit check.*
- ☒ I agree to the Terms and Conditions.*
- ☒ I agree to the Privacy Policy.*

Submit

Example of Request Finance Options at Checkout Form accepting valid input

The detailed validation rules applied to each form are provided in Appendix A.

UX Design

The Camunda forms were designed with usability and clarity in mind to reduce user error and improve the efficiency of process completion. Although the forms are primarily functional business interfaces, several UX design considerations were applied throughout their development.

Each form was divided into clearly labelled sections using headings such as Personal Information, Financial Information, and Order Information. This helped organise related fields together and reduced cognitive load by allowing users to complete the form in smaller logical stages rather than processing a large amount of information at once.

Field layouts were also designed to improve readability and workflow efficiency. Related fields, such as first and last name or city and ZIP code, were positioned side-by-side using Camunda's column layout system. This reduced unnecessary vertical scrolling and created a more natural left-to-right completion flow.

Appropriate input types were selected depending on the data being collected. For example, select boxes were used for predefined choices such as employment status, checkboxes were used for

mandatory agreements, number fields were used for financial values and quantities, and datetime fields were used for dates. This improved both usability and data consistency by guiding users toward valid input methods.

Validation rules also contributed to the user experience. Immediate validation feedback prevents users from submitting incomplete or incorrectly formatted data, reducing frustration later in the process. For example, email validation helps users quickly identify typing mistakes, while required fields clearly indicate which information must be completed before submission.

Descriptions and labels were written using straightforward language to keep the forms understandable for non-technical users. In some cases, additional contextual descriptions were included beneath fields to explain their purpose, such as indicating that a control total was intended for financial batches only.

Default values were also used where appropriate. For example, some status fields automatically populated with values such as Received, reducing manual effort and ensuring consistent process data across submissions.

Overall, the forms were designed not only to collect valid process data, but also to provide a clear, structured and predictable user experience that supports efficient completion of Camunda workflows.

Testing

Automated Testing

In addition to the manual BPMN workflow testing carried out through Camunda Operate and Tasklist, automated testing was implemented for both the Spring Boot API and the Camunda worker applications. These tests were designed to validate backend logic independently from the live Camunda environment, allowing individual components to be verified in isolation before full end-to-end process execution.

Worker Unit Testing

The Camunda workers were tested using JUnit 5 and Mockito. Each worker was tested independently by mocking the `ActivatedJob` and `JobClient` objects normally provided by Camunda at runtime. This allowed the worker logic to be executed without deploying BPMN processes or starting a Zeebe broker.

The worker tests verified that:

- process variables were read correctly from the activated job
- expected output variables were generated
- REST API calls were made correctly
- inter-process messages were published correctly
- jobs were completed successfully

For example, the `CheckToolRentalDetailsWorker` test verified that booking details such as rental rate, deposit amount, taxes, and total cost were calculated correctly from the provided job variables. Similarly, the `UpdateStockWorker` test verified that stock records were created and purchase orders updated correctly through mocked REST API calls.

Workers which communicated with external systems were tested using mocked dependencies:

- `RestTemplate` was mocked for API communication
- `MessagePublisher` was mocked for BPMN message publication
- Camunda command chains were mocked using Mockito deep stubs

This approach allowed the orchestration logic of each BPMN task to be validated without relying on live infrastructure or database state.

Repository Testing

Repository tests were implemented using Spring Boot `@DataJpaTest` with an in-memory H2 database. These tests verified that entity persistence and custom query methods behaved correctly.

Repository tests were created for:

- `ToolRepository`
- `CustomerRepository`
- `TradeCardRepository`
- `BookingRepository`
- `StockRecordRepository`
- `PurchaseOrderRepository`
- `InvoiceRepository`

- ServiceJobRepository

The tests validated:

- entity persistence
- JPA relationships
- generated SQL query behaviour
- custom repository finder methods

For example, the ToolRepositoryTest verified that findByAvailableTrue() only returned tools marked as available, while the TradeCardRepositoryTest verified that trade cards could be retrieved correctly by customer ID and card number.

Using an in-memory H2 database allowed repository logic to be tested in an isolated environment without requiring a running external database instance.

REST API Controller Testing

REST API endpoints were tested using Spring MockMvc and @WebMvcTest. These tests simulated HTTP requests directly against the controller layer without starting a full web server.

Controller tests validated:

- endpoint mappings
- HTTP request handling
- JSON serialization and deserialization
- parameter validation
- correct response payloads
- expected HTTP status codes

The ProbuildControllerTest covered a range of API functionality including:

- tool creation and availability updates
- customer creation and modification
- booking creation
- trade card point updates
- purchase order creation
- invoice generation
- service job creation

For example, tests verified that:

- POST /tools correctly created an available tool
- PUT /tools/{id}/availability updated the availability status
- POST /tradecards/{id}/points correctly increased trade card points
- POST /purchaseorders created purchase orders with associated customer data

These tests provided confidence that the API layer behaved correctly before integrating it into the wider BPMN workflows.

Testing Outcome

The automated tests complemented the manual process testing carried out within Camunda. Manual testing was still required for validating full BPMN orchestration, Tasklist forms, message correlation, and Operate behaviour. However, the automated tests significantly reduced debugging time by validating workers, repositories, and API endpoints independently from the workflow engine.

Overall, the testing approach combined:

- unit testing for worker logic
- repository testing for persistence behaviour
- controller testing for REST endpoints
- manual testing for full BPMN workflow execution

This provided coverage across both the backend implementation and the end-to-end business process behaviour of the system.

Manual Testing

In addition to automated testing we felt the need to manually test our program as there should be proper user testing to surface bugs that the automated tests could either not capture or accidentally create.

Tests done

The tests done fall under a few categories. Startup testing, worker testing and flow testing.

Startup testing aimed to confirm that when the system is launched from a clean state, each component the process engine, the API, and the worker starts correctly and in the right order, and that the repositories are populated and queryable out of the box. This included verifying that environment variables were picked up correctly, that database connections were established before any worker attempted to subscribe to a job type, and that the application failed loudly and informatively when a dependency was missing rather than silently entering a broken state. We also tested cold-start behaviour against an empty database and warm-start behaviour against a pre-populated one to make sure migrations and seed data behaved consistently.

Worker testing focused on the job workers in isolation, confirming that each worker subscribed to the correct job type, completed jobs with the expected variables, and handled failures gracefully. We deliberately triggered error paths invalid input, missing variables, downstream service unavailability to confirm that workers raised BPMN errors or incidents in the way Operate expected, rather than crashing or leaving jobs locked.

Flow testing exercised complete process instances end-to-end through Tasklist, walking through each user task as an operator would, submitting forms, and confirming that the resulting variables propagated correctly to subsequent service tasks and gateways. This was where message correlation received the most attention, since correlation keys are easy to get subtly wrong in ways that unit tests do not catch.

Testing Outcome

The manual tests complemented the automatic process testing carried out within Java. Manual testing was still required for validating full BPMN orchestration, Tasklist forms, message correlation, and Operate behaviour. However, the automated tests significantly reduced debugging time by validating workers, repositories, and API endpoints independently from the workflow engine.

Overall, the testing approach combined automated unit and integration tests in Java for fast, repeatable verification of individual components; process-level tests that exercised BPMN logic against an embedded engine; and manual exploratory testing through Tasklist and Operate to confirm that the system behaved correctly as a whole from the user's perspective. Together, these layers gave us confidence that both the individual building blocks and the orchestrated workflow were behaving as intended.

Configuration Tools and Practices

Configuration Management

Version Control and Repository Structure

Configuration management was used throughout the project to control changes, support collaboration, and maintain a reproducible development environment. Git and GitHub were used as the primary configuration management tools, with the project divided across three public repositories: Probuild-API, Probuild-worker, and Probuild-Diagrams.

This repository separation allowed the API, Camunda workers, and BPMN/form artefacts to be versioned independently while still supporting the same overall business process. The API repository followed Spring Boot conventions by separating controllers, models, and repositories. The worker repository separated individual Camunda worker classes and helper components. The diagrams repository held BPMN models, Camunda forms, and deployment support files.

The project followed a trunk-based development workflow, with most changes committed directly to the main branch. This is a recognised professional approach used in industry and was well suited to the scale and pace of this academic project, allowing rapid iteration without the overhead of formal pull request review cycles. Collaboration remained visible through the commit history, with multiple contributors, merge commits, and co-authored commits providing a clear record of joint development activity.

Version Tracking and Collaboration Evidence

The commit history shows that the project evolved incrementally over time rather than being produced as a single final upload. In the Probuild-API repository, commits show progression from the first working draft through persistent database support, README updates, endpoint changes, trade card support, the seed controller, and automated API tests. In the Probuild-worker repository, commits show staged development of Camunda workers, message publishing support, supplier integration, trade card integration, and worker unit tests. In the Probuild-Diagrams repository, commits show updates to the operational BPMN model, Camunda forms, deployment scripts, validation changes, and fixes to message correlation and workflow logic.

This history provided traceability, as individual changes could be linked to specific development tasks and earlier versions of the API, workers, and BPMN models could be recovered if a later change introduced errors. While the project did not use GitHub Issues or a formal project board to link commits to tracked requirements, the descriptive commit messages served as the primary source of change traceability throughout development.

Dependency and Environment Management

Dependency and build configuration was managed using Maven. The repositories included mvnw and mvnw.cmd wrapper scripts, allowing the applications to be built and tested consistently without requiring a globally installed Maven version on each developer's machine.

The API and worker applications used separate application.properties files to manage their runtime configuration independently. The API was configured to run on port 8081 using an H2 file-based database (jdbc:h2:file:./data/probuildddb), with JPA set to update mode so the schema evolved with the codebase without requiring manual migrations. The H2 console was also enabled at /h2-console to allow team members to inspect database state directly during development and testing. The worker was configured on port 8082 and pointed to a locally running Camunda self-managed instance via both gRPC (port 26500) and REST (port 8080). Both properties files were committed to their respective repositories, meaning any team member could clone the project and run the application in a consistent local environment without any additional manual configuration.

A .gitignore file was used to exclude generated and machine-specific files such as target/, .idea/, .vscode/, .classpath, .settings, and other IDE or build artefacts. This kept the repositories focused on source code, configuration files, tests, BPMN models, and documentation rather than temporary or environment-specific local files.

The project did not use formally separated development, test, and production environments, nor were release tags applied to mark stable versions. These are gaps, as in a larger or longer-running project they would be important additions, but for the scope of this academic project the configuration practices implemented provided a sufficient and repeatable local setup.

Documentation and Onboarding

A README was added to the API repository to support team members in setting up and contributing to the codebase. The README provided end-to-end setup instructions covering how to start the local Camunda instance, launch the API and worker applications, access the H2 database console with the correct JDBC connection string, deploy BPMN diagrams and forms via the Camunda Modeler, and observe workflow execution through Camunda Operate. It also documented how to manually populate the database using example Postman requests for endpoints such as POST /tools and GET /tools/available. This reduced setup friction for collaborators and made the repository more self-contained as a development resource.

Test and Demo Data Management

The project included configuration management for repeatable testing and demonstration data. The API included a /seed endpoint which populated the database with realistic sample data, including customers, tools, stock records, trade cards, purchase orders, invoices, bookings, and service jobs. This allowed the system to be quickly restored to a known demonstration state before running Camunda workflow tests, reduced the need for manual database setup, and made demonstrations more consistent. This was especially useful when testing BPMN workflows that depended on existing customer, tool, stock, or trade card data being present.

Automated Testing and CI Integration

Automated tests were committed into the relevant repositories as part of the configuration management process. The API tests covered repository behaviour and REST controller endpoints, while the worker tests checked Camunda job variable handling, mocked REST calls, message publication, and job completion. In addition, a GitHub Actions workflow was configured on the API repository to automatically execute the Maven test suite on each push. This meant that changes to controllers or repositories were validated without requiring manual test runs, and regressions could be caught before they propagated into shared development. Keeping both the tests and the CI configuration under version control ensured that the testing pipeline itself was reproducible and visible to all contributors.

Overall Evaluation

Overall, the project demonstrates good configuration management practice for its size and context. GitHub was used not only as storage but as a tool to track system evolution, support collaboration, document changes, and maintain separate but connected system components. The use of public repositories, descriptive commit history, Maven build files, committed application.properties configuration, .gitignore, onboarding documentation, seeded demo data, BPMN version tracking, committed automated tests, and a GitHub Actions CI pipeline provides clear evidence of structured and traceable development practice.

The main areas not implemented were GitHub Issues or a project board for requirements traceability, formal environment separation, and release tagging. These are acknowledged limitations rather than oversights, and would be natural additions in a production or team setting beyond the scope of this project.

Appendix

Appendix A - All Form Validation Use

Form	Validation Summary
<i>Select Eligible Items (£100) Form</i>	<i>Item ID given alphanumeric ID pattern; Item Name and Description min/max length constrained; Price constrained min: 0; selected items and core item fields required.</i>
<i>SelectItemsForm</i>	<i>Customer ID, Session ID and Discount Code given alphanumeric ID pattern; Unit Price, Inventory, Subtotal, Tax, Discount and Grand Total constrained min: 0; descriptions and notes capped at 255; required customer/session/product fields enforced.</i>
<i>Send Delivery Date to Customer via Email</i>	<i>Customer Email and Sender Email given email pattern; Order ID and Tracking Number given alphanumeric/tracking pattern; Tracking URL given URL pattern; Subject and Body length constrained; required notification fields enforced.</i>
<i>SendToMaintenanceForm</i>	<i>Tool ID, Employee ID, Location and Work Order fields given alphanumeric ID pattern; Staff, Technician and Signature fields given name pattern; defect descriptions and comments min/max constrained; maintenance fields required.</i>
<i>StockCountForm</i>	<i>Stock ID, Staff ID, Warehouse, Bin, SKU and Batch fields given alphanumeric ID pattern; System Quantity and Physical Quantity constrained min: 0; discrepancy notes capped at 255; verification fields required.</i>
<i>Tag Tools as in Need of Service</i>	<i>Tool ID and Reporter ID given alphanumeric pattern; Tool Name and Type length constrained; Service Issue Description min/max constrained; report and urgency fields required.</i>
<i>ToolFlaggingReport</i>	<i>Tool ID and Reporter fields given alphanumeric ID pattern; Technician Name given name pattern; Issue, Action, Maintenance and Resolution text min/max constrained; required flagging fields enforced.</i>

<i>Verify Supplier Documents</i>	<i>Contact Email given email pattern; Contact Number given phone pattern; Supplier ID, Document Reference and Verifier fields given alphanumeric ID pattern; Address, Rejection Reason and Notes capped at 255.</i>
<i>VerifySupplier DocumentsForm</i>	<i>Supplier ID, PO Number, Shipment ID and Employee ID given alphanumeric ID pattern; Name and Signature fields validated; discrepancy notes capped at 255; verification fields required.</i>
<i>WarehousePackageHandover</i>	<i>Order Number and Tracking Number given alphanumeric/tracking pattern; Customer, Staff and Driver names given name pattern; Package Description and Handling Notes min/max constrained; warehouse handover fields required.</i>
<i>WeeklyReportForm</i>	<i>Counts and Totals constrained min: 0; Percentage and Score fields constrained min: 0 / max: 100; Reviewer fields given name pattern; Report Notes, Goals and Comments capped at 255.</i>
<i>Tool_Rental_Request_Form</i>	<i>Tool Name min/max constrained; Quantity constrained min: 1; Start/End Dates and Terms Agreement required.</i>
<i>RequestFinanceOptionsAtCheckoutForm</i>	<i>Email, Phone, National ID and Postcode/ZIP patterns enforced; Income and Purchase Amount constrained min: 0; applicant, address, employment and consent fields required.</i>
<i>RescanIntoWarehouseSys</i>	<i>SKU, Serial, Warehouse, Rack, Bin, Operator, Document and Transaction fields given alphanumeric ID pattern; Quantity constrained min: 1; Notes capped at 255; rescan metadata required.</i>
<i>ReturnToolsToWarehouseAfterRepairForm</i>	<i>Tool ID, Work Order, Employee ID and Bin/Location fields given alphanumeric ID pattern; Name and Signature fields validated; Repair Description and Inspection Notes min/max constrained; warehouse return fields required.</i>

<i>Review and Approve Invoice</i>	<i>Invoice, PO, Vendor and Cost-Centre fields given alphanumeric ID pattern; Subtotal, Tax and Total Amount constrained min: 0; Comments and Rejection text capped at 255; approval fields required.</i>
<i>Review KPI Report</i>	<i>KPI Target, Actual Value and Variance constrained min: 0; Reviewer, Approver and Action Owner fields given name pattern; Summaries, Causes and Action Notes capped at 255.</i>
<i>ReviewReportsForm</i>	<i>Report ID and Subject ID given alphanumeric ID pattern; Name and Signature fields validated; Observations, Findings and Action Items min/max constrained; review and sign-off fields required.</i>
<i>Register_As_Member_Preference.form</i>	<i>Membership preference selections required; Name and Preference Notes min length enforced; Comment areas capped at 255; Email and Phone validation applied where present.</i>
<i>Attempt Membership Registration (Email).form</i>	<i>Customer Email required with RFC-style email pattern; Subject and Body capped at 255; Customer/Member identifiers required; Name validation applied.</i>
<i>BatchSubmissionForm.form</i>	<i>Batch Reference IDs given alphanumeric pattern; Quantities and Totals constrained min: 0; Notes and Comments capped at 255; submission metadata required.</i>
<i>BookToolForm.form</i>	<i>Tool Selection and Booking Dates required; Quantity constrained min: 1; Customer Email and Phone patterns enforced; Booking Notes capped at 255.</i>
<i>Check and Inspect Goods.form</i>	<i>Inspection Outcome and Inspector Details required; Inspection Comments min length enforced; Issue Descriptions capped at 255; Quantity and Damage Counts constrained min: 0.</i>
<i>Check Tool Availability in Store.form</i>	<i>Tool Identifiers and SKUs required; Stock values constrained min: 0; Inventory Reference Codes given pattern validation; Availability Notes capped at 255; location fields required.</i>

<i>CheckAndInspectGoodsForm.form</i>	<i>Inspection Checklist and Condition Status required; Stock and Inspection Quantities constrained min: 0; Notes and Issue Descriptions capped at 255; staff identification required.</i>
<i>CheckToolAvailabilityForm.form</i>	<i>Search Criteria and Tool References required; Requested Quantities constrained min: 0; SKU and Item IDs given pattern validation; Comments capped at 255; branch identifiers required.</i>
<i>CompareActualvsSystemStockForm.form</i>	<i>Actual and System Stock Counts required; Stock Quantities constrained min: 0; SKU and Barcode fields required with ID pattern validation; Discrepancy Descriptions capped at 255; Staff Verification required.</i>
<i>ConductQuarterlyReviewMeetingwithProBuildOperationsManagerForm.form</i>	<i>Meeting Date, Title and Attendee Names required; Meeting Notes min/max constrained; Action Items and Summaries capped at 255; Review Outcome fields required.</i>
<i>ConfirmBookingwithCustomerForm.form</i>	<i>Booking Confirmation fields required; Email and Phone patterns enforced; Customer identifiers required; Confirmation Notes capped at 255; Booking Reference pattern applied.</i>
<i>CreateDigitalHandoverChecklistforTools.form</i>	<i>Checklist Completion and Return/Handover Status fields required; Tool IDs/SKUs given alphanumeric ID pattern; Handover Signatures and Names validated; Handover Comments capped at 255.</i>
<i>Customer.form</i>	<i>Customer personal details and identifiers required; Email and Phone patterns enforced; Name validation applied; Addresses and Notes capped at 255.</i>
<i>Customer-Request-Tool-Hire-Form.form</i>	<i>Requested Tool and Hire Date fields required; Quantity constrained min: 1; Customer Email and Phone patterns enforced; Request Notes capped at 255.</i>

<i>DetermineBin LocationForm. form</i>	<i>Bin and Location identifiers required with warehouse/bin code pattern validation; SKU fields required; Location Notes capped at 255; Shelf/Row references numerically validated where applicable.</i>
<i>Enter Tools Back Into Stock.form</i>	<i>Returned Tool identifiers required; Stock Return Quantities constrained min: 0; Condition and Status fields required; Return Comments capped at 255.</i>
<i>Fill-In-Loan-In formation.for m</i>	<i>Loan and Customer details required; Loan Amount and Deposit constrained min: 0; Email and Phone patterns enforced; Repayment and Due Dates required; Notes capped at 255.</i>
<i>GenerateExce ptionsReport.f orm</i>	<i>Report Parameters and Output Selections required; Date Range validation applied; Report Notes capped at 255; Report Reference Codes validated.</i>
<i>GenerateInvoi ceSummary.fo rm</i>	<i>Invoice Summary fields required; Totals and Invoice Values constrained min: 0; Invoice Numbers given alphanumeric pattern; Customer/Account References required; Notes capped at 255.</i>
<i>GenerateServ iceReport.for m</i>	<i>Report Metadata and Service Date fields required; Report Descriptions and Notes capped at 255; Technician/Staff IDs validated; Service Totals constrained min: 0.</i>
<i>Repair Tool After Inspection Form</i>	<i>Tool ID and Inspection ID given alphanumeric ID pattern + max 50; Inspector Name, Requested By, Authorized By and Verified By given name pattern + max 100; signatures min 2 / max 100; Parts Cost, Labor Cost and Total Repair Cost constrained min: 0; Part Quantity and Labor Hours constrained min: 0; misc textfields capped at 255.</i>
<i>Input Delivery Locations to GPS Form</i>	<i>Recipient Name given name pattern + max 100; Recipient Phone required with phone pattern; Order ID alphanumeric max 50; City letters-only + max 50; ZIP given ZIP/ZIP+4 pattern; Latitude constrained min: -90 / max: 90; Longitude constrained min: -180 / max: 180; misc textfields capped at 255.</i>

<i>Inspect Returned Tool Condition Form</i>	<i>Customer Name and Inspector Name given name pattern + max 100; RMA Number and Order Number given alphanumeric ID pattern + max 50; Inspector Signature min 2 / max 100; misc textfields capped at 255.</i>
<i>Issue Tool to Customer with Return Date Form</i>	<i>Customer Name given name pattern + max 100; Customer ID and Tool ID alphanumeric max 50; Contact Phone and Email patterns enforced; Deposit Collected and Rental Fee constrained min: 0; Customer Signature min 2 / max 100; Issue Date constrained after yesterday; misc textfields capped at 255.</i>
<i>Log Damage Report Form</i>	<i>Reporter Name and Supervisor Name given name pattern + max 100; Employee ID and Tool ID alphanumeric max 50; Supervisor Signature min 2 / max 100; Estimated Cost constrained min: 0; misc textfields capped at 255.</i>
<i>Log Discrepancy for Auditing Purposes Form</i>	<i>Auditor Name given name pattern + max 100; Report Destination capped at 255; Numerical and Time Tolerance constrained min: 0; misc textfields capped at 255.</i>
<i>Move Items to Storage Form</i>	<i>Item ID alphanumeric max 50; Staff ID and Name capped at 255; Staff Signature min 2 / max 100; Quantity Moved constrained min: 1; misc textfields capped at 255.</i>
<i>New Form</i>	<i>Empty form with no validation components required.</i>
<i>Order Form</i>	<i>Customer, Recipient and Cardholder names given name pattern + max 100; Email and Phone patterns enforced; City letters-only + max 50; State 2-uppercase-letter pattern; ZIP given US ZIP format; Card Number constrained 13-19 digits; Expiry given MM/YY pattern; CVV constrained 3-4 digits; financial fields constrained min: 0; Quantity constrained min: 1; misc textfields capped at 255.</i>

<i>Pack Items Form</i>	<i>Order Number, Packer ID and Station ID given alphanumeric pattern; Customer Name given name pattern; Package Weight constrained min: 0; Box Count constrained min: 1; Quantity Packed constrained min: 0; Packer Signature min 2 / max 100; misc textfields capped at 255.</i>
<i>Pick Items From Stock Form</i>	<i>Order Number and Product SKU given alphanumeric pattern; Customer Name given name pattern; Quantity Required, Quantity Picked and Total Items Picked constrained min: 0; misc textfields capped at 255.</i>
<i>PlaceOrderForm</i>	<i>Skipped due to unreadable file/I-O error.</i>
<i>Prepare Improvement Plan Form</i>	<i>KPI ID alphanumeric max 50; KPI Owner and Approver Name given name pattern; Plan Prepared By capped at 255; Target Achievement Date constrained after yesterday; misc textfields capped at 255.</i>
<i>Prompt User for Information / Payment Info Form</i>	<i>Full Name and Card Name given name pattern + max 100; Email required with pattern; Phone pattern applied; Card Number constrained 13-19 digits; Expiry given MM/YY pattern; CVV digit-only pattern enforced; City letters-only + max 50; ZIP pattern applied; misc textfields capped at 255.</i>
<i>Prompt Payment Info Form</i>	<i>Cardholder Name given name pattern + max 100; Card Number constrained 13-19 digits; Expiry given MM/YY pattern; CVV digit-only pattern enforced; City letters-only + max 50; ZIP pattern applied; Email required with pattern; Phone pattern applied; misc textfields capped at 255.</i>
<i>Prompt User Info Form</i>	<i>Shipping/Billing Names given name pattern + max 100; Shipping/Billing City letters-only + max 50; Shipping/Billing State constrained to 2 uppercase letters; ZIP fields given US ZIP format; Email required with pattern; Phone pattern applied; Card Number constrained 13-19 digits; Expiry given MM/YY pattern; CVV digit-only pattern enforced; misc textfields capped at 255.</i>

Purchase Items in Store Form *First Name, Last Name and Cardholder Name given name pattern; Email required with pattern; Phone pattern applied; Card Number constrained 13-19 digits; Expiry given MM/YY pattern; financial fields constrained min: 0; Customer Signature min 2 / max 100; misc textfields capped at 255.*

Put Back to Stock Form *Tool ID and Inspection ID given alphanumeric ID pattern + max 50; Inspector Name given name pattern; Staff Name capped at 255; Staff Signature min 2 / max 100; Quantity Returned constrained min: 1; misc textfields capped at 255.*

Receive Tool Return Form *Tool ID alphanumeric max 50; Returner Name given name pattern + max 100; Contact Phone pattern applied; Returner and Receiver signatures min 2 / max 100; misc textfields capped at 255.*

Record Batch in Client Portal FixPro Form *Batch ID, Service Record ID, Client ID and Asset ID given alphanumeric ID pattern + max 50; Client Name, Submitted By and Client Printed Name given name pattern + max 100; Total Hours constrained min: 0; Part Quantity constrained min: 1; Client Signature min 2 / max 100; misc textfields capped at 255.*

Customer (Membership Registration) *Email required with standard email pattern; Phone pattern enforced; City letters-only + max 50; State constrained to 2 uppercase letters; ZIP given US ZIP format; Username constrained min 4 / max 30 with alphanumeric+_- pattern; Password constrained min 8 / max 128 with uppercase, digit and special character required; Card Number constrained 13-19 digits; CVV constrained 3-4 digits; Cardholder Name letters/spaces/hyphens only; misc textfields capped at 255.*

Customer Request - Tool Hire Form *Email required with pattern; Phone pattern applied; Customer Signature min 2 / max 100; misc textfields capped at 255.*

Determine Bin Location Form *Product ID constrained min 1 / max 50 with alphanumeric pattern; Shipment ID and PO Number given alphanumeric pattern; Manager Name letters-only pattern; Confirmation Signature min 2 / max 100; Expiration Date constrained after today; misc textfields capped at 255.*

Enter Tools Back Into Stock *Tool ID and Employee ID given alphanumeric ID pattern + max 50; Employee Name and Receiver Name given letters-only pattern; Receiver Signature min 2 / max 100; misc textfields capped at 255.*

Fill-In Loan Information *SSN/ITIN constrained to ###-##-#### or 9-digit pattern; Email and Phone required with pattern; Annual Income and Loan Amount constrained min: 0; Date of Birth constrained before today; Digital Signature min 2 / max 100; misc textfields capped at 255.*

Generate Exceptions Report *Report Title, Process Definition Key, Activity ID, Error Code and Job Type capped at 255; misc textfields capped at 255.*

Generate Invoice Summary *Currency constrained to exactly 3 uppercase letters; Total Invoice Amount, Tax, Amount Due, Payments and Balance constrained min: 0; Invoice Number given alphanumeric ID pattern; misc textfields capped at 255.*

Generate Service Report *Requester Name given letters-only pattern + max 100; Report Title required with max 255; misc textfields capped at 255.*

Handover to Customer Form *Customer Name, Staff Name and Printed Name given letters-only pattern + max 100; Customer Phone pattern applied; Staff ID and Order Number alphanumeric max 50; ID last-4-digits field constrained to exactly 4 digits; Quantity Expected and Quantity Handed Over constrained min: 0; signatures min 2 / max 100; misc textfields capped at 255.*

Identify Failing KPIs and Root Causes Form *KPI ID alphanumeric max 50; KPI Owner, Reported By and Reviewer Name given letters-only pattern; Target Value and Actual Value required; misc textfields capped at 255.*

<i>Batch Submission Form</i>	<i>Client Batch Ref capped at 255; Record Count constrained min: 0; Control Total constrained min: 0; Client ID and Submitted By capped at 255; misc textfields capped at 255.</i>
<i>Book Tool Form</i>	<i>Full Name and Cardholder Name given letters-only pattern + max 100; Email required with pattern; Phone pattern applied; Card Number constrained 13-19 digits; CVV constrained 3-4 digits; Expiration Date given MM/YY pattern; Digital Signature min 2 / max 100; Quantity constrained min: 1; misc textfields capped at 255.</i>
<i>Check and Inspect Goods</i>	<i>PO Number and Supplier Name given ID/name pattern; Inspector Name given letters-only pattern + max 100; Expected, Received, Accepted and Rejected Quantities constrained min: 0; Inspector Signature min 2 / max 100; misc textfields capped at 255.</i>
<i>Check Tool Availability in Store</i>	<i>ZIP constrained to US ZIP format; City letters-only + max 50; State constrained to 2 uppercase letters; Notification Email given email pattern; Search Radius constrained min: 1 / max: 100; misc textfields capped at 255.</i>
<i>Check and Inspect Goods Form (v2)</i>	<i>Inspection ID alphanumeric max 50; Inspector Name and Supplier Name given name pattern; Expected and Received Quantities constrained min: 0; Warehouse Manager Signature min 2 / max 100; misc textfields capped at 255.</i>
<i>Check Tool Availability Form (Service Request)</i>	<i>Full Name given letters-only pattern + max 100; Email required with pattern; Phone pattern applied; Quantity constrained min: 1; misc textfields capped at 255.</i>
<i>Compare Actual vs System Stock Form</i>	<i>Product SKU and Barcode given alphanumeric ID pattern; Staff ID alphanumeric max 50; Staff Name and Supervisor Name given name pattern; Counter and Supervisor Signatures min 2 / max 100; Actual Physical Count constrained min: 0; misc textfields capped at 255.</i>
<i>Conduct Quarterly Review Meeting with</i>	<i>Percentage KPI fields constrained min: 0 / max: 100; financial fields constrained min: 0; Meeting Title capped at 255; misc textfields capped at 255.</i>

*ProBuild
Operations
Manager
Form*

*Confirm
Booking with
Customer
Form* *Contact Email and Phone required with pattern; Customer Name given letters-only pattern + max 100; financial readonly fields constrained min: 0; misc textfields capped at 255.*

*Create Digital
Handover
Checklist for
Tools* *Tool ID and Handover IDs given alphanumeric pattern + max 50; Handover Names given letters-only pattern + max 100; signatures min 2 / max 100; Battery Quantity constrained min: 0; misc textfields capped at 255.*